COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	August 18, 2026	**DATE SUBMITTED:**	August 29, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

Screenshot	`76 28 6 92 98 37 22 80 27 1 52 36 92 26 43 15 92 14 78 76 8 29 21 27 21 34 69 70 56 43 37 74 68 18 22 34 79 30 72 6 78 5 94 72 26 23 63 7 3 27`
Observations	This is a randomly generated dataset designed to be searched using the searching techniques below.

Table 6-1. Data Generated and Observations.

Code

searchingAlgo.h

```
4   template <typename T>
5   void linearSearchArr(int arrSize, T dataFind, T list[]) {
6
7       // Step 1: I := 0
8       int i = 0;
9
10      // Repeat while I <= n
11      while (i < arrSize) {
12
13          //  If (item = data[i])
14          if (dataFind == list[i]) {
15
16              //  Print "Searching is successful"
17              std::cout << "Searching is successful\n";
18              return;
19          }
20          i++;
21      }
22
23      // Print "Searching is Unsuccessful"
24      std::cout << "Searching is unsuccessful" << std::endl;
25  }
```

main.cpp

```
linearSearchArr(max_size, userInput, dataset);
```

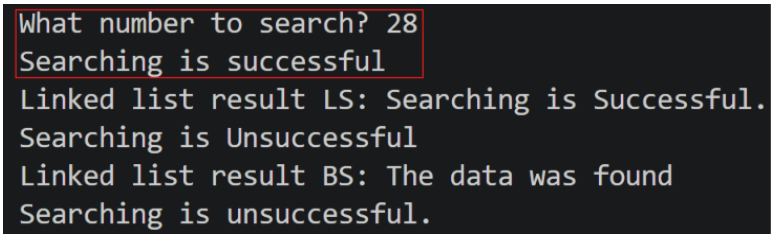
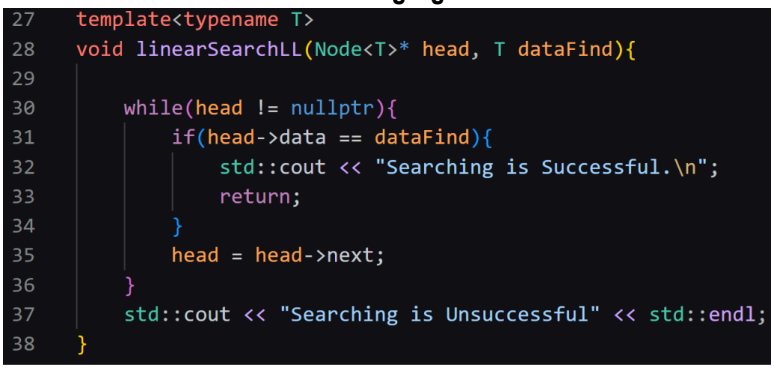
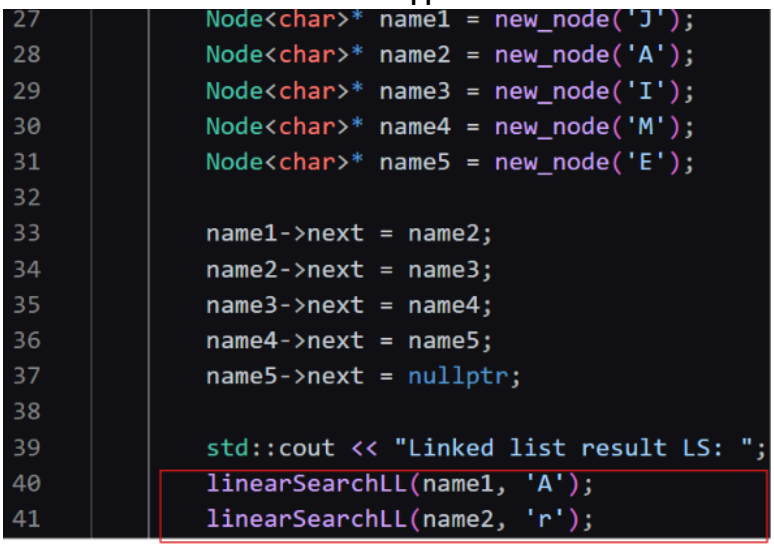
Output	
Observations	This is responsible for checking array elements sequentially. Searching 28 returns “Searching is successful” as soon as it finds the matching value. If it is not on the list, the function traverses the entire array before printing “Searching is unsuccessful”

Table 6-2a. Linear Search for Arrays

Code	**searchingAlgo.h**  **main.cpp** 

```
27  template<typename T>
28  void linearSearchLL(Node<T>* head, T dataFind){
29
30      while(head != nullptr){
31          if(head->data == dataFind){
32              std::cout << "Searching is Successful.\n";
33              return;
34          }
35          head = head->next;
36      }
37      std::cout << "Searching is Unsuccessful" << std::endl;
38  }
```

```
27          Node<char>* name1 = new_node('J');
28          Node<char>* name2 = new_node('A');
29          Node<char>* name3 = new_node('I');
30          Node<char>* name4 = new_node('M');
31          Node<char>* name5 = new_node('E');
32
33          name1->next = name2;
34          name2->next = name3;
35          name3->next = name4;
36          name4->next = name5;
37          name5->next = nullptr;
38
39          std::cout << "Linked list result LS: ";
40          linearSearchLL(name1, 'A');
41          linearSearchLL(name2, 'r');
```

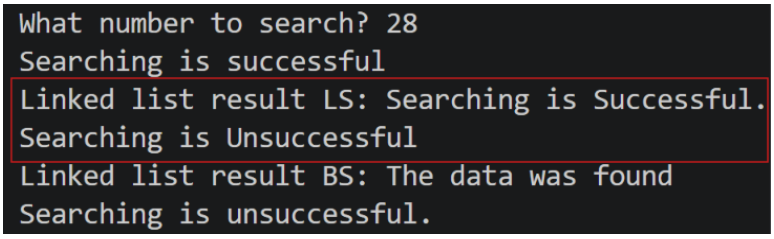
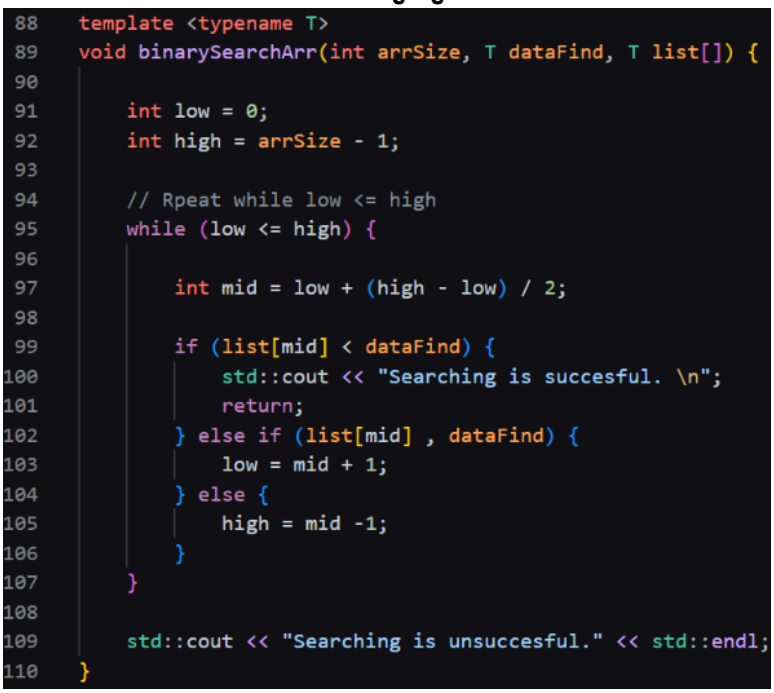
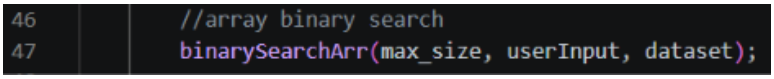
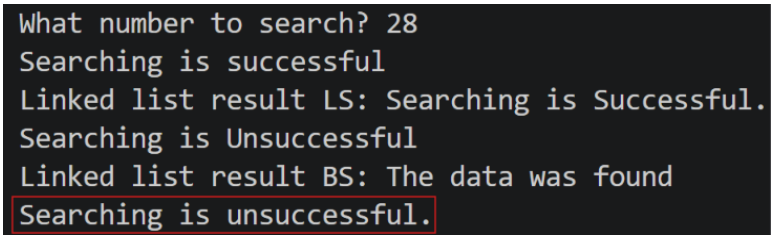
Output	
Observations	The traversal starts at name1 and moves pointer by pointer. After searching for character 'A' it returns successfully. Searching starting from name2 for 'r' returns unsuccessful because it is not in the node list.

Table 6-2b. Linear Search for Linked List

Code	**searchingAlgo.h**  **main.cpp** 
Output	

```
88  template <typename T>
89  void binarySearchArr(int arrSize, T dataFind, T list[]) {
90
91      int low = 0;
92      int high = arrSize - 1;
93
94      // Rpeat while low <= high
95      while (low <= high) {
96
97          int mid = low + (high - low) / 2;
98
99          if (list[mid] < dataFind) {
100             std::cout << "Searching is succesful. \n";
101             return;
102         } else if (list[mid] , dataFind) {
103             low = mid + 1;
104         } else {
105             high = mid -1;
106         }
107     }
108
109     std::cout << "Searching is unsuccesful." << std::endl;
110 }
```

```
46          //array binary search
47          binarySearchArr(max_size, userInput, dataset);
```

Observations	This uses low and high indices to divide the search range in half each iteration. It compares the middle element, which is list[mid], to dataFind. This reduces the time to search a lot.

Table 6-3a. Binary Search for Arrays

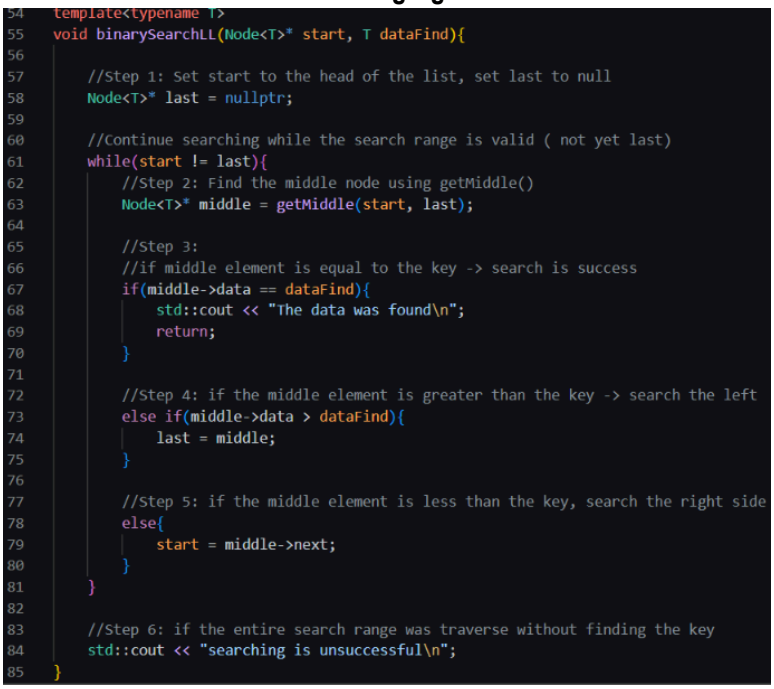
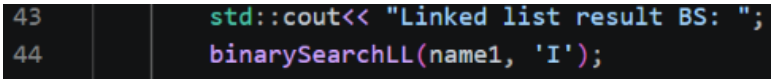
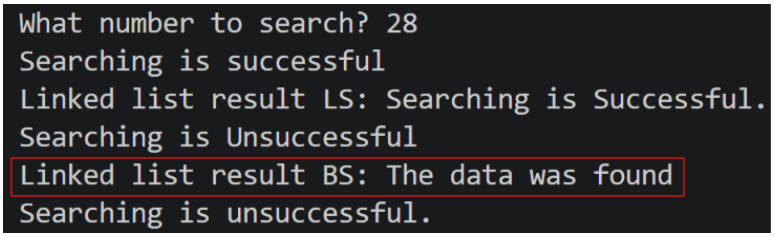
Code	**searchingAlgo.h**  **main.cpp** 
Output	
Observations	This uses (getMiddle()) to locate the midpoint node. Passing name1 and searching for 'I' returns "The data was found".

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

supplementarySearchingAlgo.h

```cpp
template <typename T>
int linearSearchArrCount(int arrSize, T dataFind, T list[]) {

    int comparisons = 0;

    for (int i = 0; i < arrSize; i++) {
        comparisons++;
        if (list[i] == dataFind) {
            std::cout << "Found at index " << i << ". Comparisons: " << comparisons << std::endl;
            return comparisons;
        }
    }

    std::cout << "Not found. Comparisons: " << comparisons << std::endl;
    return comparisons;
}
```

```cpp
template <typename T>
int linearSearchLLCount(Node<T>* head, T dataFind) {

    int comparisons = 0;

    while (head != nullptr) {
        comparisons++;
        if (head->data == dataFind) {
            std::cout << "Found. Comparisons: " << comparisons << std::endl;
            return comparisons;
        }
        head = head->next;
    }

    std::cout << "Not found. Comparisons: " << comparisons << std::endl;
    return comparisons;
}
```

supplementaryMain.cpp

```cpp
    int list1[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size1 = sizeof(list1) / sizeof(list1[0]);

    Node<int>* head = new_node(list1[0]);
    Node<int>* temp = head;
    for (int i = 1; i < size1; i++) {
        temp->next = new_node(list1[i]);
        temp = temp->next;
    }

    //Problem 1

    std::cout << "Array approach: ";
    linearSearchArrCount(size1, 18, list1);

    std::cout << "Linked list approach: ";
    linearSearchLLCount(head, 18);
```

Output

```
Array approach: Found at index 1. Comparisons: 2
Linked list approach: Found. Comparisons: 2
```

Answer:
2 key comparisons are needed for both the array and linked list implementations.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

supplementarySearchingAlgo.h

```cpp
template <typename T>
int countRepeatArr(int arrSize, T k, T list[]) {

    int count = 0;

    for (int i = 0; i < arrSize; i++) {
        if (list[i] == k) {
            count++;
        }
    }

    std::cout << "Value " << k << " appears " << count << " times in the array." << std::endl;
    return count;
}

template <typename T>
int countRepeatLL(Node<T>* head, T k) {

    int count = 0;

    while (head != nullptr) {
        if (head->data == k) {
            count++;
        }
        head = head->next;
    }

    std::cout << "Value " << k << " appears " << count << " times in the linked list." << std::endl;
    return count;
}
```

supplementaryMain.cpp

```cpp
    //Problem 2
    std::cout << "Array approach: ";
    countRepeatArr(size1, 18, list1);

    std::cout << "Linked list approach: ";
    countRepeatLL(head, 18);
```

Output

```
Array approach: Value 18 appears 2 times in the array.
Linked list approach: Value 18 appears 2 times in the linked list.
```

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

supplementarySearchingAlgo.h

```
73  template <typename T>
74  void binarySearchArr(int arrSize, T dataFind, T list[]) {
75
76      int low = 0;
77      int high = arrSize - 1;
78
79      // Rpeat while low <= high
80      while (low <= high) {
81
82          int mid = low + (high - low) / 2;
83
84          if (list[mid] < dataFind) {
85              std::cout << "Searching is succesful. \n";
86              return;
87          } else if (list[mid] , dataFind) {
88              low = mid + 1;
89          } else {
90              high = mid -1;
91          }
92      }
93
94      std::cout << "Searching is unsuccesful." << std::endl;
95  }
```

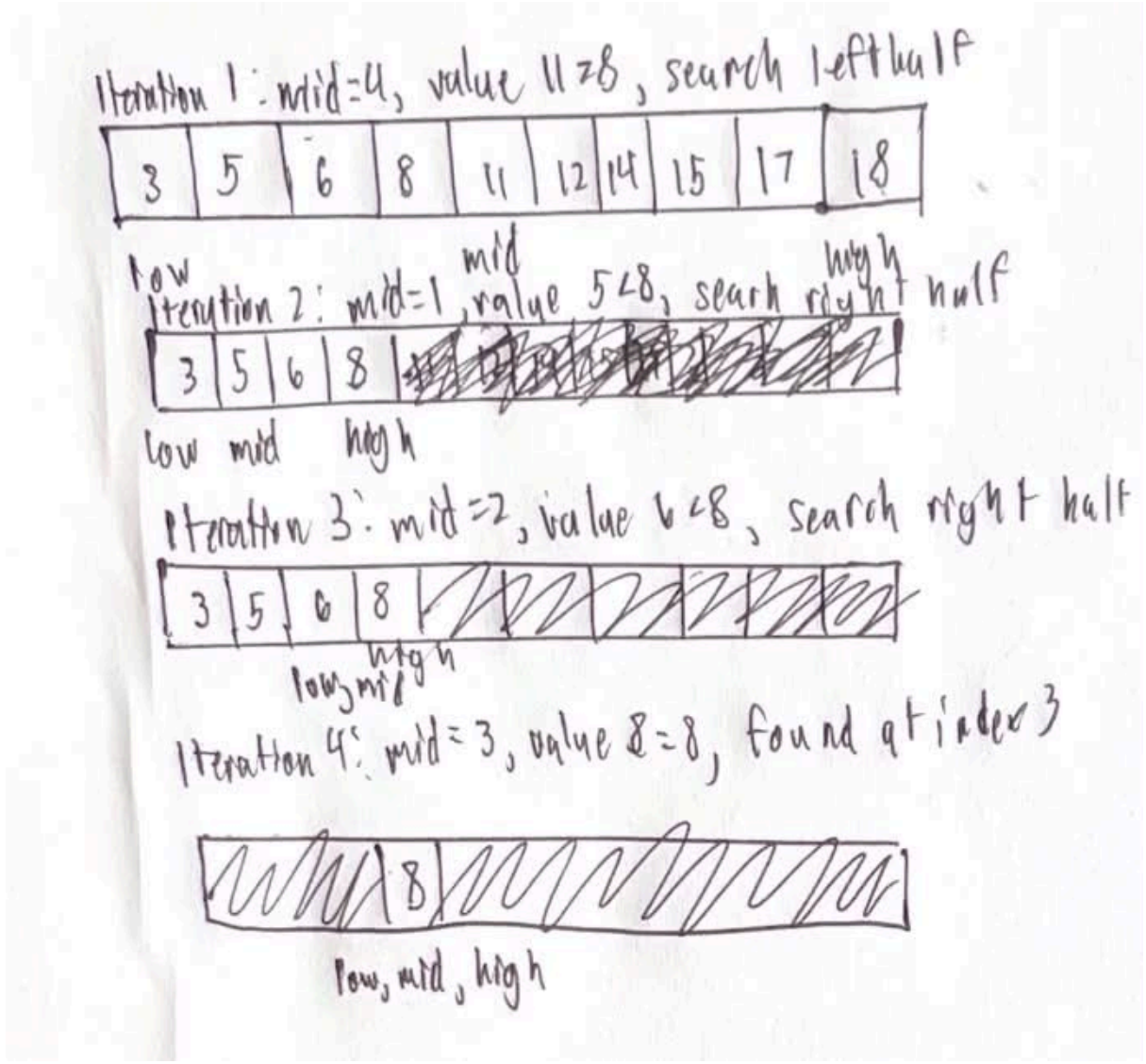
supplementaryMain.cpp

```
33      int list2[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
34      int size2 = sizeof(list2) / sizeof(list2[0]);
35
36      binarySearchArr(size2, 8, list2);
```

Output

Searching is unsuccesful.

Answer:



Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

supplementarySearchingAlgo.h

```cpp
template <typename T>
void binarySearchArrRecursive(T list[], int low, int high, T dataFind) {

    if (low > high) {
        std::cout << "Searching is unsucceful." << std::endl;
        return;
    }

    int mid = low + (high - low) / 2;

    std::cout << "Checking index " << "(" << mid << "): " << list[mid] << std::endl;

    if (list[mid == dataFind]) {
        std::cout << "Searching is succesful. Found at index " << mid << std:: endl;
        return;
    } else if (list[mid] < dataFind) {
        binarySearchArrRecursive(list, mid + 1, high, dataFind);
    } else {
        binarySearchArrRecursive(list, low, mid - 1, dataFind);
    }

}
```

supplementaryMain.cpp

```cpp
    //Problem 4
    std::cout << std::endl;
    binarySearchArrRecursive(list2, 0, size2 - 1, 8);



    Node<int>* curr = head;
    while (curr != nullptr) {
        Node<int>* next = curr->next;
        delete curr;
        curr = next;
    }
}
```

Output

```
Checking index (4): 11
Searching is succesful. Found at index 4
```

Conclusions:

This activity demonstrates implementations of linear and binary search using arrays and linked lists. I learned that linear search is simple and works on data in any order, checking each element one by one until it finds a match or reaches the end. However, Binary search is different and only works correctly on sorted data; it has to be managed by sorting the array before searching or by manually entering linked list values in ascending order. I also learned that arrays allow direct index access, so binary search can go straight to any mid index, while linked lists require a helper function like getMiddle, using the two-pointer such as slow and fast technique just to get to the middle node. This is important to know because algorithms require different implementations depending on the structure. Overall, I correctly implemented all four algorithms in the supplementary activity. If I could find a way to improve my understanding of linear and binary search, I would be practicing this implementation in real-world problems. Then I could know that I understood the two concepts if successfully implemented.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, recalling of bubble sort**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Grammarly for correction and ChatGPT for recalling the concept of bubble sorting**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.